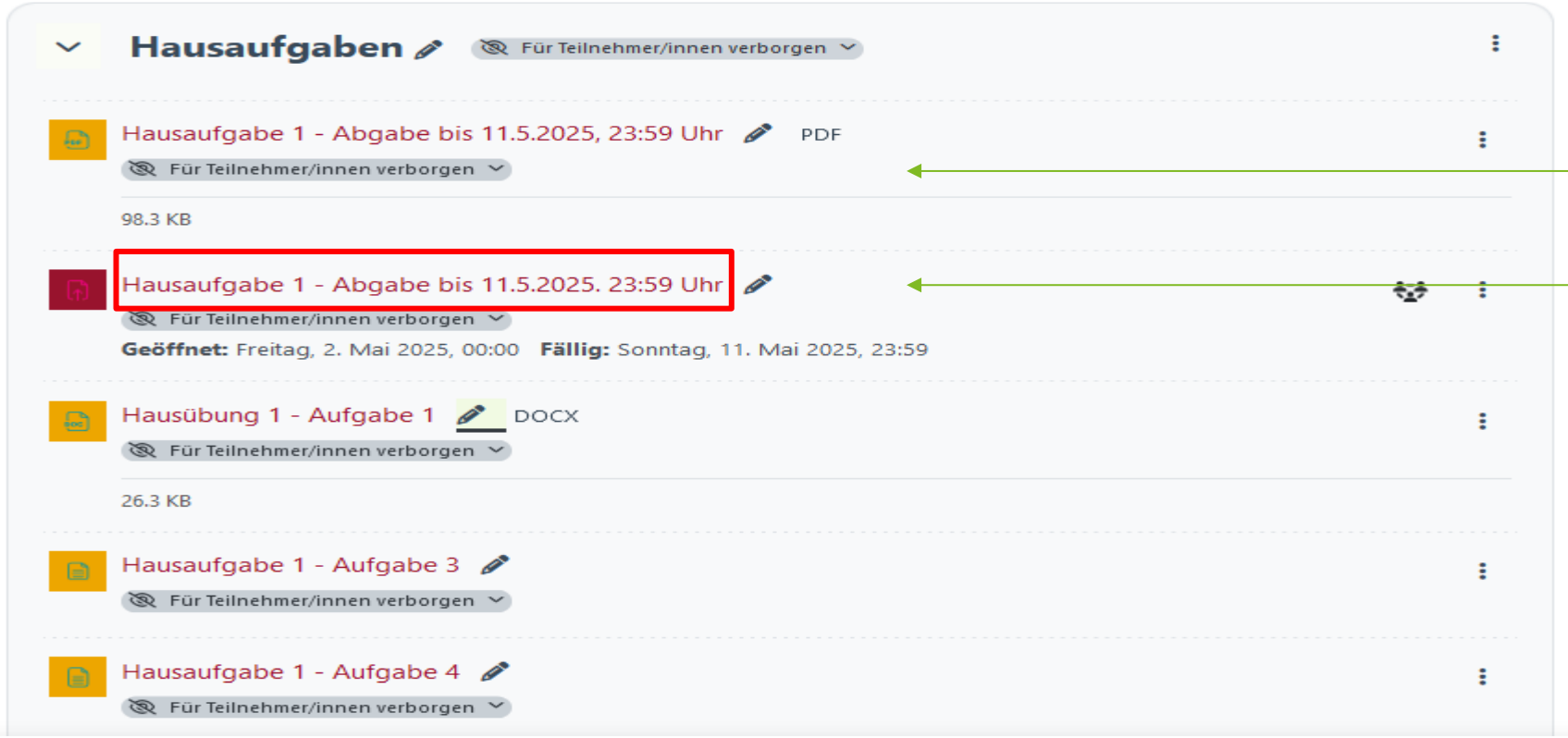
1.	Organisatorisches	4
2.	Einleitung	21
3.	HTML	46
4.	**CSS**	**70**
5.	Frameworks	89
6.	HTTP	106
7.	Internet	126
8.	TypeScript	178
9.	HTML+CSS+Typescript	224
10.	Web Services	235
11.	REST-Services	258

Hinweise zur ersten Hausübung



Aufgabenstellung

Abgabe

- Die Abgabe ist NUR in Gruppen möglich.
- Hinweise zum Abgabeformat finden Sie in der Aufgabestellung.
- Es wird insgesamt drei Hausaufgaben geben, die Sie alle drei bestehen müssen.

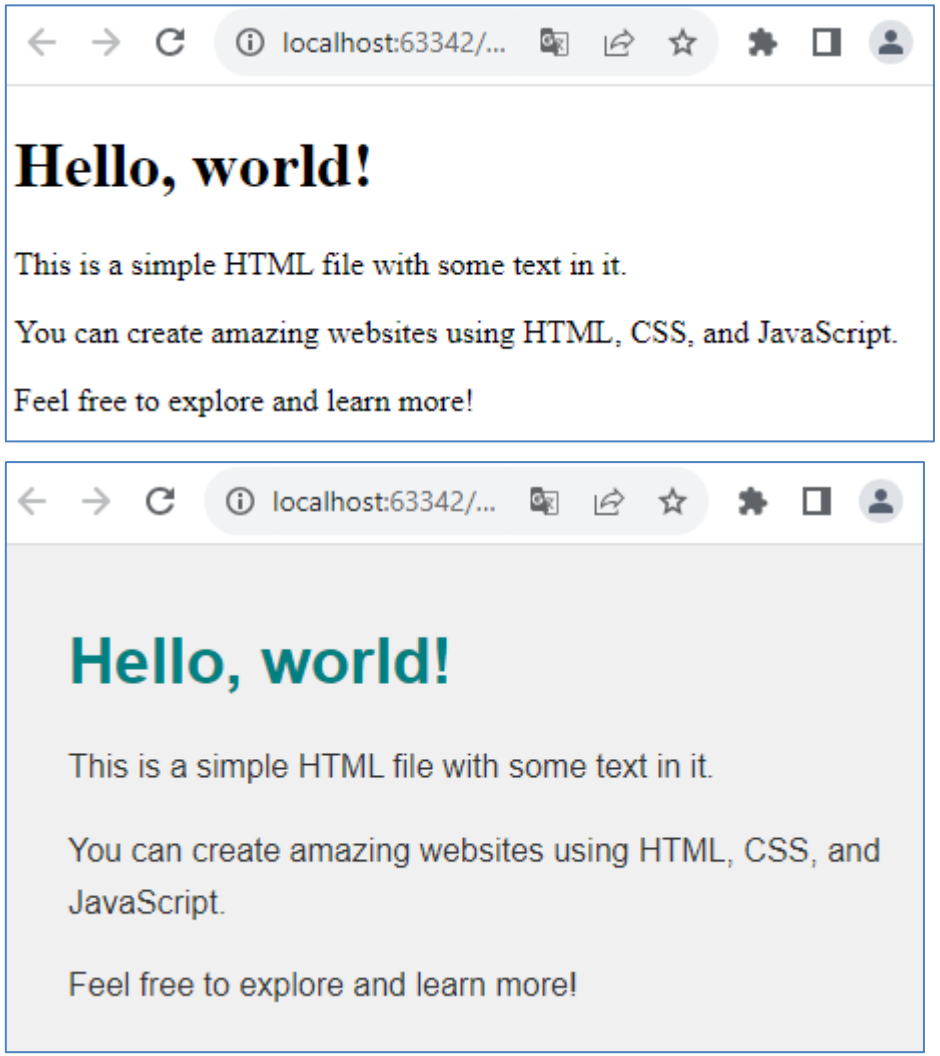
Cascading Style Sheets (CSS)

Eine HTML-Datei

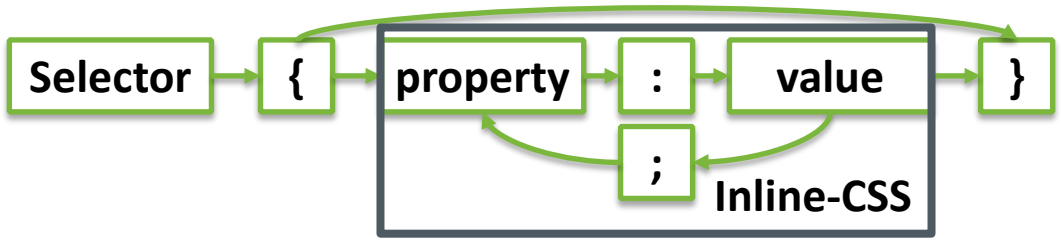
```
<!DOCTYPE html>
<html>
<head>
    <title>Simple HTML File</title>
</head>
<body>
    <h1>Hello, world!</h1>
    <p>This is a simple HTML file with some text in
it.</p>
    <p>You can create amazing websites using HTML, CSS,
and JavaScript.</p>
    <p>Feel free to explore and learn more!</p>
</body>
</html>
```

Information und Darstellung sind unabhängig voneinander!

Beliebiges Aussehen



Syntax



Inline-CSS

```
<h1 style="font-size: 32px; font-family: Arial, sans-serif; color: lightslategrey;">Eine etwas andere
Überschrift</h1>
```

Eine etwas andere Überschrift

Ausgelagert in den head :

```
<head>
 <title>Titel</title>
  <style>
   h1 {font-size: 32px; font-family: Arial, sans-serif; color: lightslategrey }
   h2 {font-size: 24px; font-family: Arial, sans-serif; color: darkslategrey }
  </style>
</head>
```

Eine etwas andere Überschrift

Kleiner!

Selektoren

- Ein **Selektor** bestimmt ein **Muster**, welches ein **Element erfüllen** muss, **um ausgewählt** zu werden.

- **Arten von einfachen Selektoren:**
  - Universalselektor `*` passt zu allen Elementen
  - Typselektor `table` passt zu Elementen von diesem Typ
  - Id-Selektor `#mainTitle` passt zu Elementen mit diesem **Id**-Attribut
  - Klassenselektor `.content` passt zu Elementen mit dieser **Klasse** (**class**-Attribut)

CSS-Dateien

content.html

```
<!DOCTYPE html>
<html lang="de">
<head>
  <link rel="stylesheet"    href="styles.css">
   …
</head>

<body>
…
</body>

</html>
```

styles.css

```
body {background-color: lavenderblush}
h1 {font-size: 24px; color: #f0f0f0; background: #7399ae
}
#mainTitle {font-size: 36px }
.content {
     margin: 10px auto;
     width: 800px;
     border: 5px solid floralwhite;
     background-color: white;
     padding: 10px;
     font-size:16px;
 }
```

1. Inline-CSS gilt nur für das jeweilige Element – siehe vorvorige Seite Mitte.
2. CSS in der HTML-Datei gilt für alle Elemente in der Datei – siehe vorvorige Seite unten.
3. CSS in eigener Datei gilt für alle html-Dateien, die diese CSS-Datei verwenden – diese Seite.

Anwendbarkeit der Selektoren

content.html

```
<head>
  <link rel="stylesheet"    href="styles.css">
<body>
   Der Body hat die Klasse &lt;body&gt;

   <h1 id="mainTitle“ class="content") >
      Überschrift (id mainTitle, Klasse content) 36px
   </h1>
   <div class="content">
      Hier könnte Ihre Werbung stehen! (Klasse content) 16px
   </div>
   <h1>Zweite Headline (Klasse h1 und body) 24px </h1>
</body>
```

styles.css

```
body {background-color: lightsalmon}

h1 {font-size: 24px; color: green }

.content {
     background-color: white;
     font-size: 16px;
  }

#mainTitle {font-size: 36px }
```

Es wird der Style angewandt, der am spezifischsten ist.

1. ID-Selektor (im Beispiel: #mainTitle)
2. Klassen-Selektor (im Beispiel: .content)
3. Typ-Selektoren (im Beispiel: h1 und body)
4. Universeller Selektor (*)

Kombinator-Selektoren

Kombinator-Selektoren verbinden einfache Selektoren:

- Oder
  Bsp. `div, p { background-color:blue }` — Alle `div's` und alle Paragraphen
- Und
  Bsp. `div.content { background-color:blue }` — Alle `div's` der Klasse `content`
- Nachfolger-Selektor (descendant) (`Leerzeichen`)
  Bsp. `div p { background-color:blue }` — Alle Paragraphen innerhalb von `div`s
- Kind-Selektor (child) (`>`)
  Bsp. `div>p { background-color:green }` — Alle Paragraphen direkt innerhalb von `div`s
- Adjazenter Geschwister-Selektor (sibling) (+)
  `div+p { background-color:yellow }` — Alle Paragraphen direkt nach einem `div`
- Genereller Geschwister-Selektor (sibling) (~)
  `div+p { background-color:yellow }` — Alle Paragraphen nach einem `div`

Pseudo-Klassen (1/2)

Pseudo-Klassen werden verwendet, um den Style in Abhängigkeit vom Zustand des Elementes zu ändern.

- Syntax:
  ```
  selector:pseudo-class { property: value; }
  ```

- Beispiele:
  - Hover: .roterFont:hover { color: red;}
  - Focus: .graueEingabe:focus { background-color: darkgray; }
  - Links unvisited/visited: a:link { color : blue; } a:visited { color: deeppink; }

- http://www.w3schools.com/Css/css_pseudo_classes.asp

Pseudo-Klassen (2/2)

Es werden allgemeine Pseudo-Klassen und Pseudo-Klassen für Links unterschieden:

Allgemeine Pseudo-Klassen

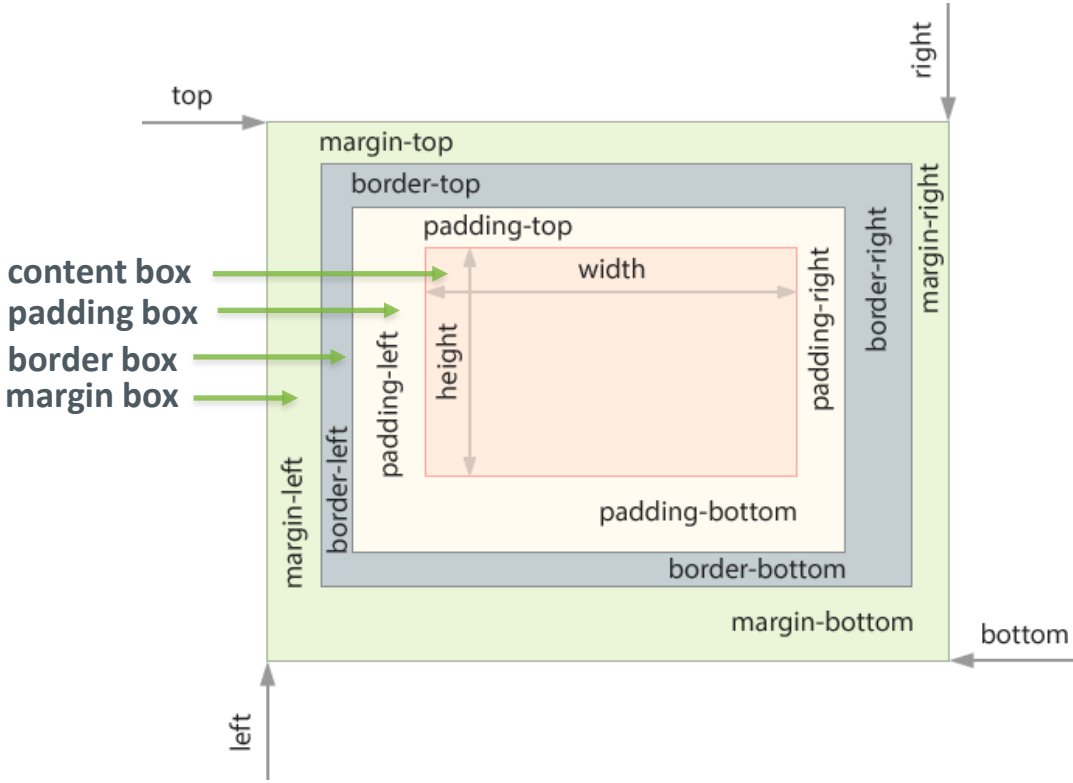
Pseudo-Klasse	Beschreibung
`:hover`	Mouse-over hover
`:focus`	Elemente, die im Fokus sind (typischerweise Felder in einer Form)
`:active`	Elemente, auf die geklickt wird
`:first-child`	Erstes Unterelement
`:last-child`	Letztes Unterelement
`:nth-child()`	Unterelemente bezogen auf ihre Position (z.B. odd/even)

Link-Pseudo-Klassen

Element	Beschreibung
`:link`	Unbesuchte Links
`:visited`	Besuchte Links
`:hover`	Mouse-over hover
`:active`	Links, auf die geklickt wird

Box-Modell

- Die Gesamtgröße, die das Elternelement bereitstellen muss, ergibt sich aus der Summe von **margin**, **border**, **padding** und {**width**, **height**}.
- Größe der Box bezieht sich auf die **Border-Box**
- **border**- und **padding**-Angaben verändern die content-box
- Negative Werte sind ebenfalls möglich



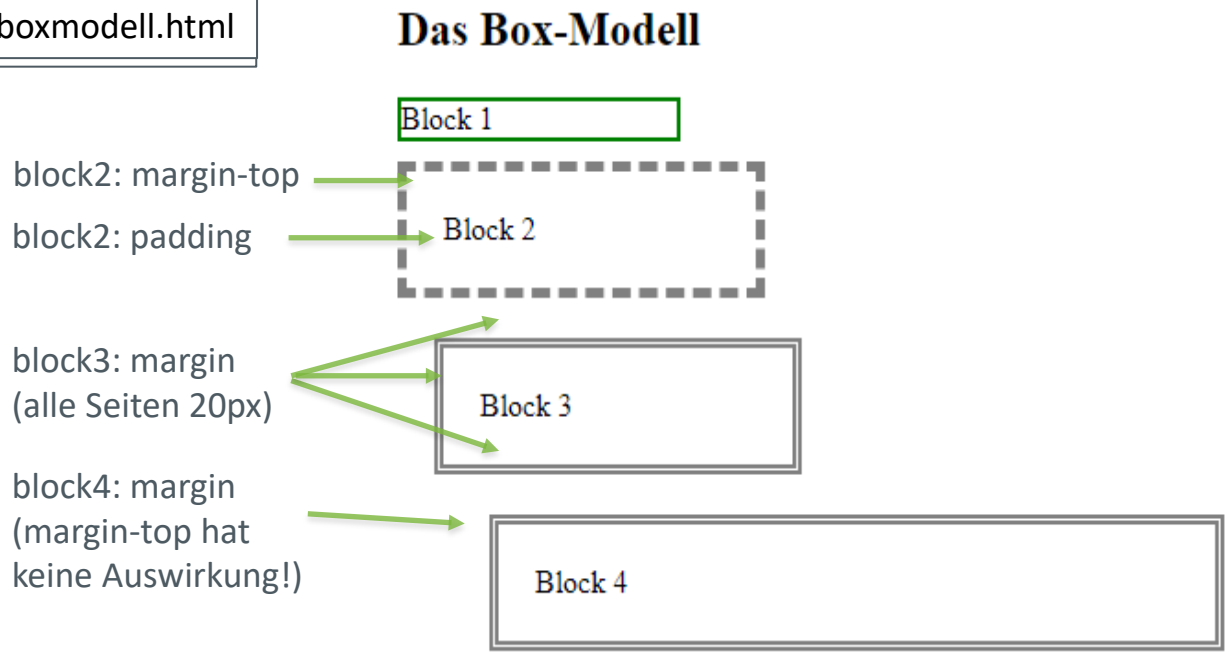
Beispiel

```css
.block1 {
    width: 150px;
    padding: 0px;
    border: 2px solid green;
    margin: 0;
}
.block2 {
    width: 150px;
    padding: 20px;
    border: 5px dashed grey;
    margin-top: 10px;
}
.block3 {
    width: 150px;
    padding: 20px;
    border: 5px double grey;
    margin: 20px;
}
.block4 {
    width: 350px;
    padding: 20px;
    border: 5px double grey;
    margin: 2px 0 0 50px;
}
```

boxstyles.css

```html
<head>
  <link rel="stylesheet"
href=„boxstyles.css">
</head>
<body>
  <h2>Das Box-Modell</h2>
  <div class="block1">Block 1</div>
  <div class="block2">Block 2</div>
  <div class="block3">Block 3</div>
  <div class="block4">Block 4</div>
</body>
```

boxmodell.html



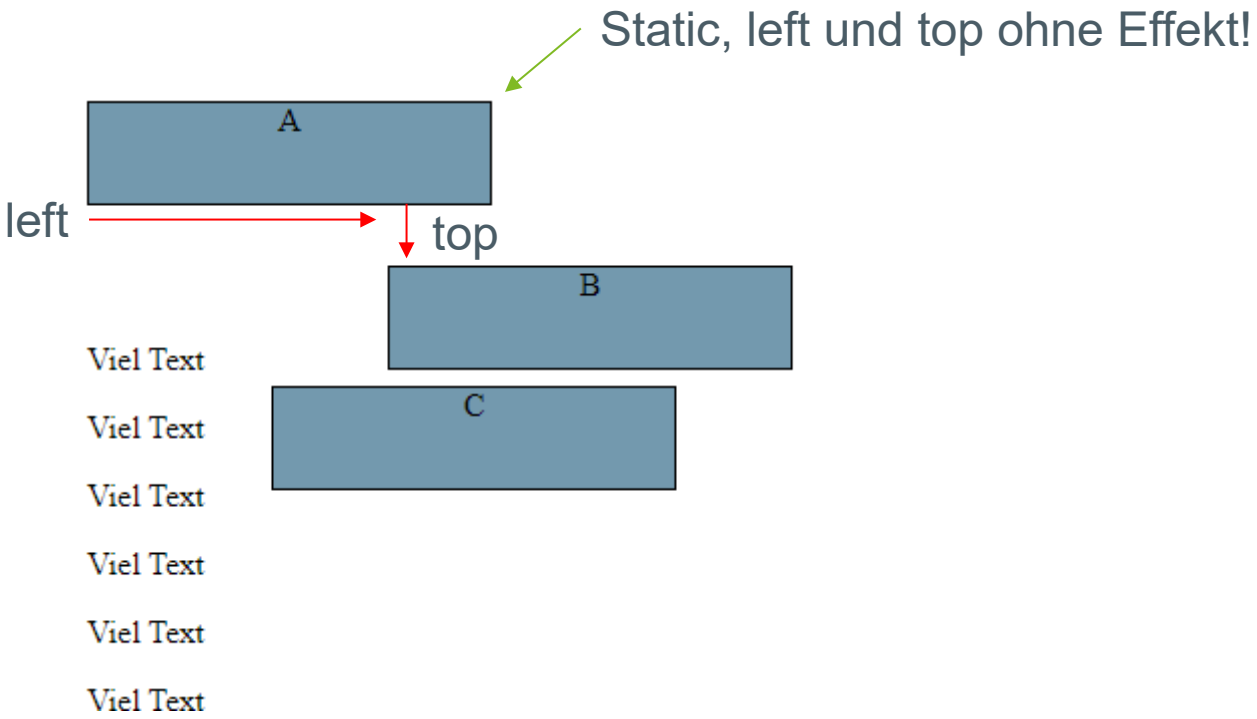
Layouting - Position

- Die Position eines Elementes wird durch die Eigenschaften {**top**, **left**, **right**, **bottom**} und {**static**, **relative**, **absolute**, **fixed**} definiert
- Default ist **position**: **static**
- **static** Element wird im normalen Fluss platziert
  {top, right, bottom, left} haben keine Auswirkungen
- **relative** Element wird relativ zur normalen Position verschoben
  und hat keine Auswirkungen auf andere Elemente
- **absolute** Element wird aus dem normalen Fluss gelöst und kann andere Elemente verdecken
  {top, left, right, bottom} bezieht sich auf übergeordnete Elemente, für die
  position: {fixed, relative, absolute} definiert wurde
- **fixed** Element wird aus dem normalen Fluss gelöst und seine Position orientiert sich am Viewport (Sicht).
  Beim Scrollen bleibt es an seiner Position im Viewport und beim Drucken erscheint es auf jeder Seite.

Beispiel

```
div {  width: 200px;
       height: 50px;
       background-color: #7399ae;
       border: 1px solid black;
       text-align: center;
    }
.a { position: static; left:150px; top: 30px;}
.b { position: relative; left:150px; top: 30px;}
.c { position: fixed; left:100px; top: 150px;}
```

```
<div class="a">A</div>
<div class="b">B</div>
<div class="c">C</div>
<p>Viel Text</p>
<p>Viel Text</p>
```



Was ändert sich beim Scrollen?

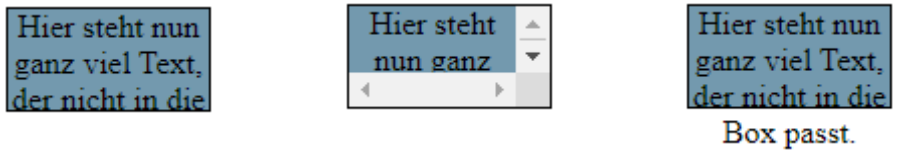
Layouting - overflow

- Definiert, was passieren soll, falls der Inhalt die Größe des Elternelements übersteigt.

```
overflow: {visible, hidden, scroll, auto}
```

- Standard: `overflow: visible`

- **`visible`** Inhalt wird außerhalb der Box angezeigt.
- **`hidden`** Inhalt, der außerhalb der Box liegt, ist unsichtbar.
- **`scroll`** Inhalt, der außerhalb der Box liegt, ist unsichtbar. Scroll-Bar wird hinzugefügt, um zum Rest zu gelangen.
- **`auto`** Browserabhängig. Meist wird auf **`scroll`** zurückgegriffen.



hidden scroll visible

Layouting - float

- Erlaubt Elementen, andere Elemente (z.B. Text um Bilder) umfließen zu lassen.

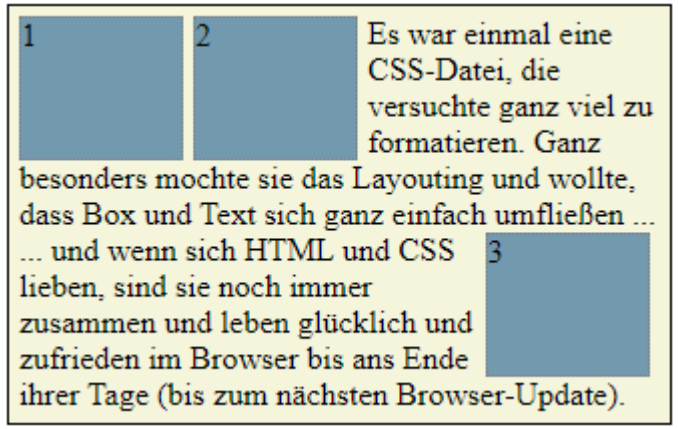
  `float: {none, left, right}`

- Ein mit **`float`** versehenes Element positioniert sich ganz {rechts, links} im Elternelement und lässt auf der anderen Seite Platz für andere Elemente.

  → andere Elemente = Block und Inline-Elemente

```
.wrapper { width: 320px;
  min-height: 180px;
  border: 1px solid black;
  background-color: beige;
  padding: 5px; }
.box { height: 70px;
  width: 80px;
  border: 1px dotted gray;
  background-color: #7399ae;
  margin-right: 5px; }
.right { float: right; }
.left { float: left; }
```

```
<div class="wrapper">
  <div class="box left">1</div>
  <div class="box left">2</div> Es war einmal eine
CSS-Datei ...
  <div class="box right">3</div> und wenn sich HTML
und CSS lieben, sind sie noch immer zusammen.
</div>
```



Media-Queries

Durch **Media Queries** können Bedingungen definiert werden, welche vom Ausgabemedium erfüllt werden müssen, um korrekte Stylesheets zu laden

- Angabe im HTML Head (Einbinden der Stylesheet-Datei):
  - Lade immer

```
<head>
  <link rel="stylesheet" href="styles.css">
</head>
```

  - Lade nur für Anzeigegeräte (`print` für Drucker)

```
<head>
  <link rel="stylesheet" href="styles.css" media="screen">
</head>
```

- In Stylesheets:

```
@media (min-width: 200px) and (max-width: 600px)
```

Es gibt Abfragen zu: Auflösung, Größe, Ausgabemedium, Farbunterstützung, Seitenverhältnissen, Orientierung, …

http://www.w3schools.com/css/css3_mediaqueries.asp